

Distributed Backend Systems – Production-Level Guide

1. Overview

This document is a comprehensive guide for designing scalable, fault-tolerant, high-performance backend systems.

Topics Covered: - Load balancers & high availability

- Database sharding & large data management
- Redis caching & cache consistency
- Kafka & event-driven microservices
- Maintaining consistency without compromising availability
- Microservice patterns & production-ready architectural best practices

Goal: Provide a 7+ years experience blueprint for modern distributed systems.

2. Load Balancers & High Availability

2.1 User Request Flow

- Users send requests to multiple load balancers (LBs) for redundancy.
- DNS resolves to multiple LBs (round-robin or weighted).
- LBs distribute traffic to application servers.

Failover scenario: If one LB fails, traffic is automatically routed to another LB. Ensures high availability and no single point of failure.

2.2 Session Management & Scaling

- Horizontal scaling of servers behind LB.
 - Sessions stored in Redis or DB, avoiding sticky sessions to maintain availability.
-

3. Database Sharding & Large Data Handling

3.1 Sharding Basics

- Sharding = splitting data across multiple databases based on a sharding key.
- Example: user_id as sharding key → users distributed across multiple shards.

3.2 User Authentication Flow

1. Extract user_id from request.
2. Lookup shard via Shard Registry.
3. Query the corresponding shard to authenticate the user.

3.3 Large User Data

- Split data horizontally if too large for one shard.
- Use indices for fast lookup.
- Avoid cross-shard joins; prefer denormalization or event-driven updates.

3.4 Aggregations & Multi-Field Grouping in MongoDB

```
db.orders.aggregate([
  { $group: { _id: { userId: "$userId", status: "$status" }, total: { $sum:
"$amount" } } }
])
```

- \$unwind converts arrays into multiple documents.
 - Complex joins across shards are expensive; avoid in production.
-

4. Caching Strategies with Redis

4.1 Purpose of Caching

- Reduce DB load
- Improve response latency
- Smooth traffic spikes

4.2 Cache-aside (Lazy Caching) Pattern

```
const cacheKey = `user:${userId}`;
let user = await redis.get(cacheKey);

if (!user) {
  user = await db.users.findById(userId);
  await redis.set(cacheKey, JSON.stringify(user), "EX", 300); // TTL 5 min
}

return JSON.parse(user);
```

- Cache hot paths only
- Use TTL & LRU eviction policies

- Granular keys → better memory utilization
- Cache user-specific data only if reused

4.3 Pagination & Partial Caching

- Cache pages of results instead of entire tables.
- Cache partial objects, not full entities.

Example: orders:user=123:page=1 → 20 orders

5. Kafka & Event-Driven Architecture

5.1 Overview

- Kafka is a distributed, durable, high-throughput event streaming platform.
- Enables asynchronous, decoupled microservices.
- Events are replayable, helping maintain consistency and cache updates.

5.2 Kafka Concepts

Concept	Meaning
Topic	Stream of messages
Partition	Ordered subset of topic
Producer	Writes messages
Consumer	Reads messages
Offset	Position in partition
Consumer Group	Multiple consumers sharing topic

5.3 Kafka Production Example

Producer Example:

```
await producer.send({
  topic: "order_created",
  messages: [{ key: "user_123", value: JSON.stringify({ orderId: "0123" }) }]});
```

Consumer Example:

```
await consumer.run({
  eachMessage: async ({ message }) => {
```

```
const event = JSON.parse(message.value.toString());
await processInventory(event.orderId); // idempotent update
}
});
```

Advantages: decouples services, replayable events, supports cache invalidation.

6. Maintaining Consistency Without Compromising Availability

6.1 CAP Theorem

- Cannot have strong consistency + availability + partition tolerance simultaneously.
- Real systems: strong consistency where business-critical, eventual consistency elsewhere.

6.2 Consistency Patterns

Pattern	Description
Event-driven	Services emit events to update dependent systems asynchronously
Read-your-writes	User sees own updates immediately
Idempotency	Prevent duplicate operations during retries
Versioning / Optimistic Locking	Avoid lost updates without distributed locks
Saga / Compensating Actions	Handle distributed failures gracefully
Cache-aside	Cache only after DB write; avoid blocking writes

6.3 User-Dependent Data & Caching

- Cache user-specific data if reused (profile, permissions, dashboard)
- Avoid caching low-reuse, volatile, or very large datasets
- Rule: cache hot & reusable, not just shared data

6.4 Cache Consistency Strategies

Strategy	Description
TTL	Let cache expire automatically
Write-through / write-back	Update cache on DB change
Event-based invalidation	Use Kafka events to invalidate/update cache
Partial caching	Cache only essential fields

6.5 Production Flow Example (Task Flow System)

1. User creates task → DB write
2. Emit `task_created` event → Kafka
3. Cache updated in Redis
4. Notification Service → sends emails
5. Analytics / reporting updated asynchronously
6. User sees task immediately (read-your-writes)
7. System remains available even if some services fail

Patterns used: Event-driven, cache-aside, Saga/compensating actions, idempotency, TTL, read-your-writes, optimistic locking.

7. Patterns Summary for Production

Category	Patterns / Practices
Load Balancing	Multiple LBs, DNS failover, session storage in Redis
DB & Sharding	Shard by key, single source of truth, avoid cross-shard joins
Caching	Cache-aside, TTL, LRU, user-specific caching if reused
Event-driven	Kafka topics, partitions, consumer groups, replayable events
Consistency	Eventual consistency, idempotency, read-your-writes, sagas
Query Optimization	Pagination, partial projections, avoid full table loads

8. Mental Model for Senior Developers

- Availability first → never block user requests
 - Consistency where required → strong for payments, eventual for dashboards
 - Eventual everywhere else → async via Kafka / microservices
 - Cache selectively → hot data, reused data, user-specific if reused
 - Redis + Kafka + DB → backbone of production systems
-

9. Interview-Ready Statements

1. **Consistency vs Availability:** “We prioritize availability, enforce strong consistency only for business-critical paths, and rely on eventual consistency elsewhere.”
2. **Caching:** “We cache selectively based on reuse patterns and hot paths, with TTL and key granularity to prevent memory bloat.”

3. **Event-driven Architecture:** "Kafka decouples services, supports replayable events, and ensures eventual consistency without blocking users."
 4. **User-Specific Caching:** "We cache user-specific data only if reuse is high; caching depends on access patterns, not ownership."
-

This single document consolidates: - Load balancing

- Sharding & DB queries
- Redis caching & cache strategies
- Kafka & event-driven microservices
- Maintaining consistency without compromising availability
- Microservice patterns & production-ready architecture